

An Experimental Evaluation of TCP Congestion Control Algorithms

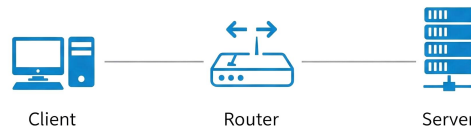
EECS 122 Final Project • Nicholas Carpenedo • Rohin Shanker • Allison Dana

RESEARCH GOAL

Compare performance across multiple algorithms:

- **BBR3 + CUBIC**

Environment: Ubuntu VM and iperf3

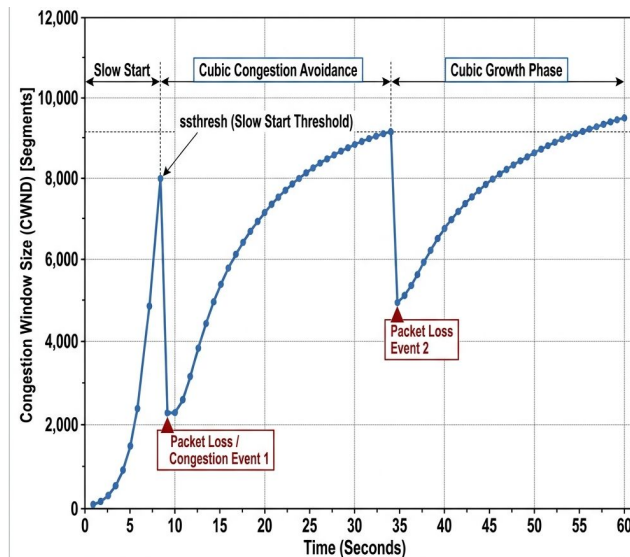


Deep Dive: CUBIC Congestion Control

The default TCP congestion control algorithm for Linux kernels

CORE PRINCIPLES

- **Loss-Based:** CUBIC relies on packet loss as the primary signal for network congestion.
- **Cubic Function:** Uses a cubic function of time since the last congestion event to scale the window.
- **Stability:** Designed to maintain stability and fairness in high-bandwidth, high-latency networks.



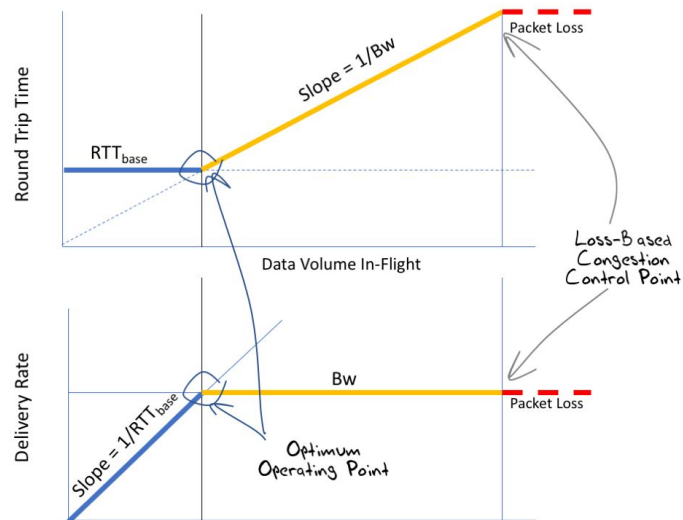
Data is simulated by Google NetSim v1.3 | All features of Cubic TCP are preserved

Deep Dive: BBR Congestion Control

Bottleneck Bandwidth and Round-trip propagation time (BBR) v3

CORE PRINCIPLES

- **Model-Based:** Unlike Reno/CUBIC, BBR builds an explicit model of the network path.
- **BDP Targeting:** Sends just enough data to fully use the connection without overloading it.
- **Loss Resilience:** Ignores random packet loss, preventing unnecessary throughput backoff.
- **Efficiency:** Minimizes queueing delay by not filling buffers at the bottleneck.



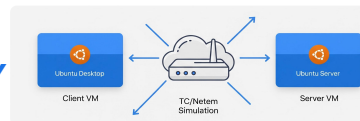
Experimental Process & Environment

Emulating real-world network dynamics using dual Ubuntu VM clones

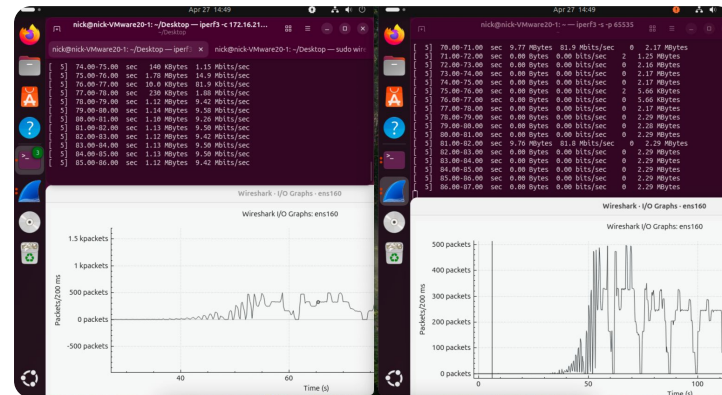
Methodology

- 01 Setup & Cloning:** Installed Ubuntu 25.10 on VMWare Fusion; created a Linked Clone for client/server pair.
- 02 Link Simulation:** Utilized `tc` and `netem` to tweak latency, jitter, and loss variables.
- 03 Traffic Generation:** Executed `iperf3` tests between VMs to generate traffic using a specific algorithm
- 04 Analysis:** Collected JSON data to interpret trade-offs between throughput and latency control.

NETWORK TOPOLOGY



Host-Only Networking: Isolated environment to ensure no external traffic interference.



Experiments Conducted

1

Packet Loss vs. Throughput

- Varied packet loss rate while keeping delay constant
- Measured impact on throughput and stability

2

Delay vs. Throughput

- Varied network delay with zero packet loss
- Evaluated performance under increasing RTT

3

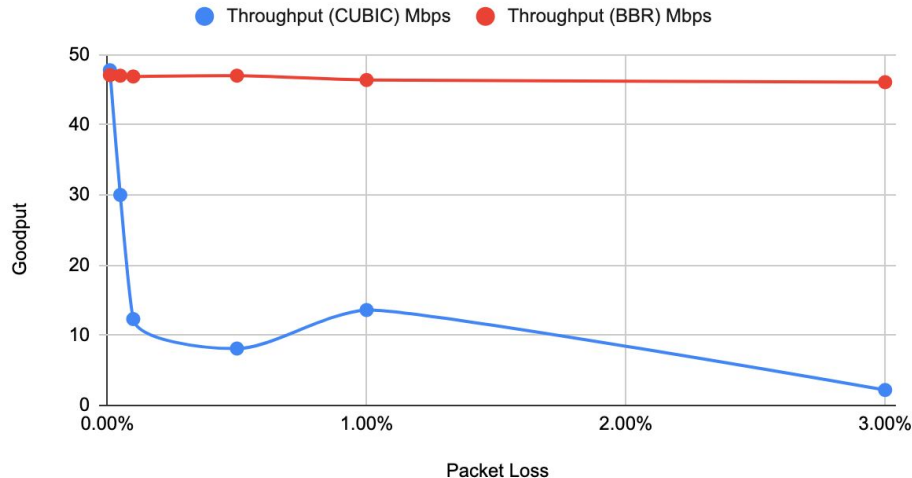
Real-World Link Emulation

- Simulated representative network conditions
- Terrestrial, Low Earth Orbit Satellite, Geostationary Satellite conditions

Experimental Results: Packet Loss Analysis

Comparative resilience performance of BBR3 and CUBIC performance under network congestion

Throughput (CUBIC) Mbps and Throughput (BBR) Mbps



Key Observations

- **BBRv3** maintains a high, stable throughput of ~45 Mbps regardless of packet loss.
- **CUBIC** throughput drops significantly as packet loss increases.

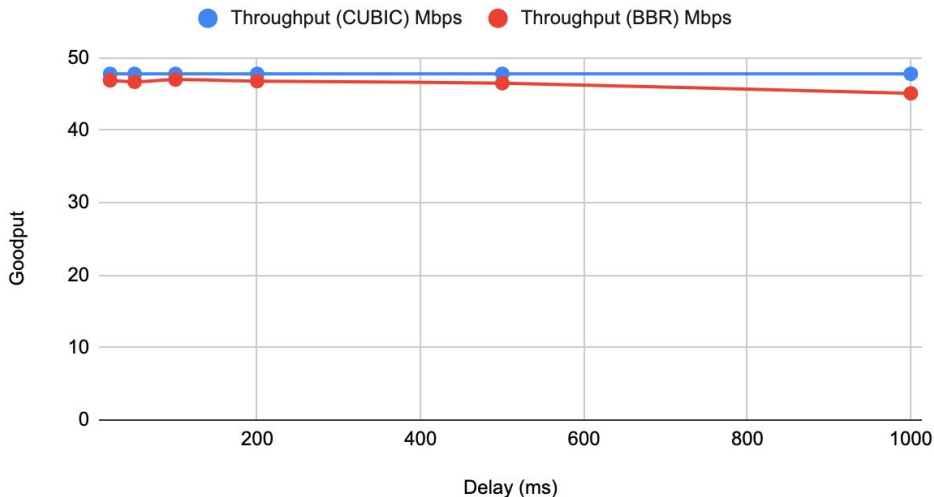
Experiment Conditions:

- **Delay:** 50ms
- **Jitter:** 0ms
- **Loss:** Varied
- **Rate:** 50 Mbps
- **Queue Depth:** $2 \times \text{BDP}$

Experimental Results: Delay Analysis

How CUBIC and BBR respectively respond to greater network delay

Throughput (CUBIC) Mbps and Throughput (BBR) Mbps



Key Observations

- BBR slightly outperforms CUBIC
- We hypothesize that BBR would outperform CUBIC in shallow buffer conditions

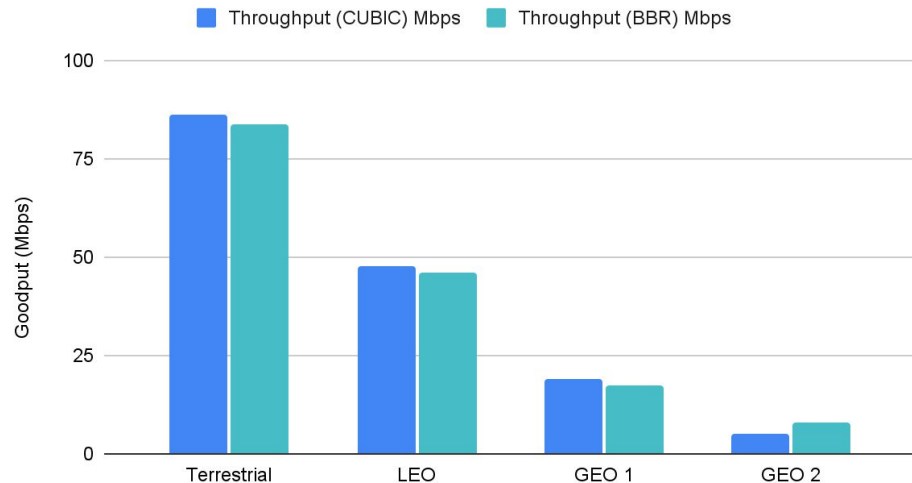
Experiment Conditions:

- **Delay:** Varied
- **Jitter:** 0ms
- **Rate:** 50 Mbps
- **Loss:** 0%
- **Queue Depth:** $2 \times \text{BDP}$

Experimental Results: Common Links

How CUBIC and BBR behave in various representative link conditions

Throughput vs Link Type



Motivation

Naval vessels rely on LEO/GEO SATCOM with high delay/loss

Experiment Conditions:

- **Loss Correlation:** 25%
- **Queue Depth:** $2 \times \text{BDP}$

Category	Delay (ms)	Jitter (ms)	Rate (Mbps)	Loss
Terrestrial	20	2	90	0.01%
LEO	80	8	50	0.30%
GEO 1	500	50	20	1%
GEO 2	1000	100	10	3%

Automation: Planned Analysis Pipeline

Python scripts will parse iperf3 JSON logs, aggregate repeated trials, and generate standardized plots for throughput, retransmissions, RTT, and congestion-window behavior.



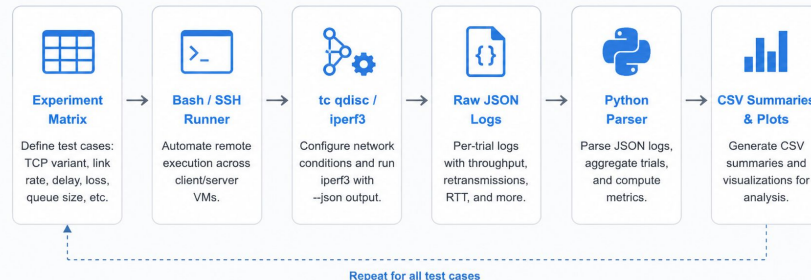
Automation Commands & Scripts

Leverage bash scripting to orchestrate multi-VM experiments sequentially without intervention.

- **ssh**: Remote command execution across client/server nodes.
- **iperf3 --json**: Outputs raw data in machine-readable format for parsing.
- **tc qdisc**: Sets bottleneck rate, delay, loss, and queue behavior for trials.



Automated Data Graphing Pipeline



Python **matplotlib** and **pandas** scripts automatically ingest JSON logs to generate publication-ready plots immediately after test completion.

Future Research

Potential avenues for extending congestion control evaluation



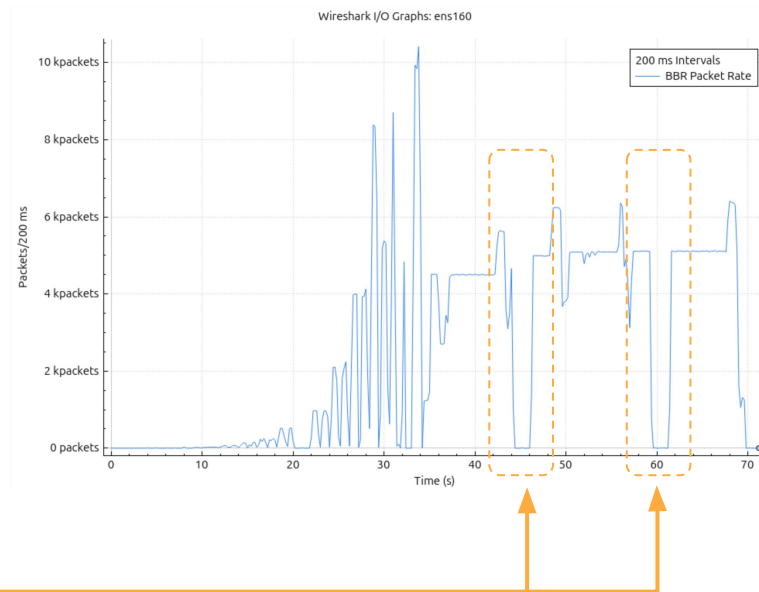
Additional Test Cases

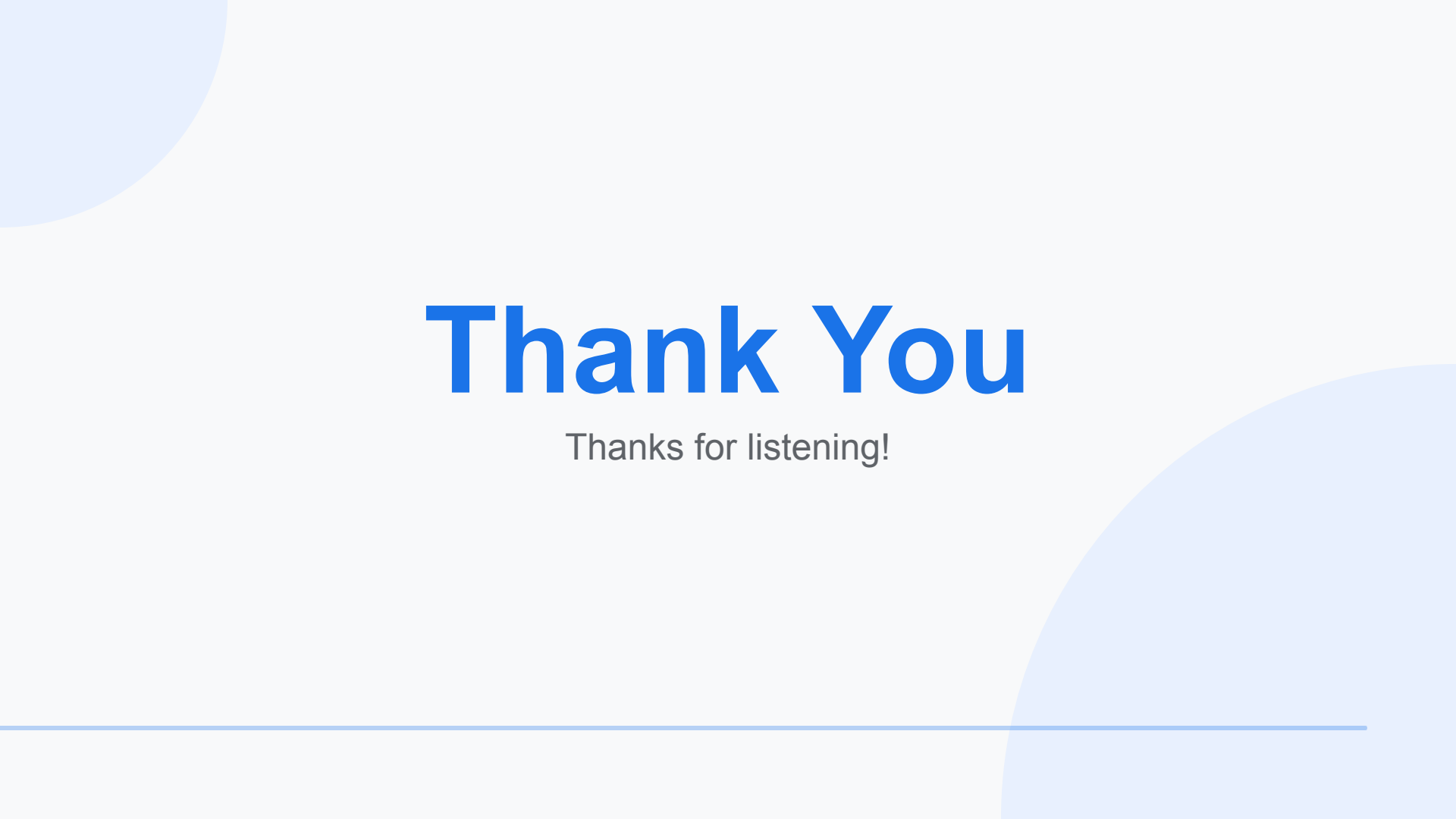
- Deep vs. Shallow Buffer Conditions
- Additional Congestion Control Algorithms
- Short duration flows
- Multiple competing flows



Investigating Anomalies

- Investigate source of burstiness in iperf
- Investigate periodic stalls in high latency BBRv3 flows





Thank You

Thanks for listening!

